

Foundations of Transfer Learning and Ensemble Learning

Before examining the implementation of the model, it is important to understand the theoretical foundations behind two key ideas used in this project: **transfer learning** and **ensemble learning**. These techniques allow deep learning models to achieve high performance even when the dataset is relatively small compared to massive datasets used to train modern neural networks.

1. Transfer Learning (Core Idea)

Training a deep neural network entirely from scratch requires enormous amounts of data and computational power. When a neural network starts training with **randomly initialised weights**, it must learn everything from the beginning, including basic visual concepts such as edges, textures, and shapes.

This process is extremely inefficient when the dataset is small. To solve this problem, modern deep learning systems often rely on **transfer learning**, which allows a model to reuse knowledge learned from a previous task.

A model trained on **ImageNet** has already learned fundamental visual features from millions of images. These features include patterns that are common across almost all visual tasks, such as edges, contours, and object structures. Because these features are universal, they can be reused when training a model for a new task.

Instead of training a network from random initialisation, we begin with pretrained weights and then adjust them slightly for the new dataset.

The difference between these two approaches can be illustrated as follows:

Training From Scratch

Random weights

↓

Learn edges

↓

Learn textures

↓

Learn shapes

↓

Learn objects

↓

Final classifier

Transfer Learning

Pretrained weights

↓

Already knows edges

↓

Already knows textures

↓

Already knows shapes

↓

Fine-tune for the new task

↓

Final classifier

Because the model already understands visual structure, it only needs to adapt to the **specific categories of the new dataset**.

This significantly reduces training time and often improves performance.

2. Feature Hierarchy in Convolutional Neural Networks

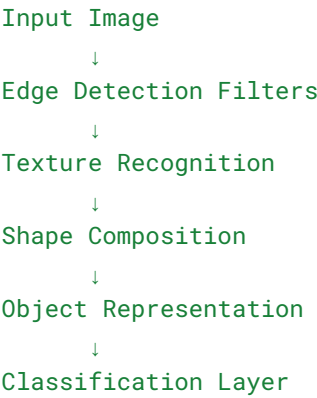
Convolutional Neural Networks (CNNs) hierarchically learn visual information. Each layer in the network extracts progressively more abstract features from the input image.

Early layers detect simple patterns, while deeper layers capture complex semantic structures.

The learning hierarchy typically looks like this:

Layer Depth	What It Learns
Early Layers	edges, gradients, colors
Middle Layers	textures, repeated patterns
Deep Layers	object parts
Final Layers	full object concepts

This hierarchy can be visualised as a processing pipeline.



The important insight here is that **early layers are general**, while **later layers are task-specific**.

Therefore, in transfer learning:

- Early layers are **kept unchanged**
- Later layers are **fine-tuned or replaced**

This allows the model to adapt to new tasks while retaining its foundational visual knowledge.

3. Mathematical Representation of Transfer Learning

A neural network can be represented mathematically as a function that maps an input to an output.

$$y = f(x; W)$$

Where:

- x represents the input image
- W represents the model's parameters or weights
- y represents the predicted output

In traditional training from scratch, the weights are randomly initialised:

$$W_{\text{random}}$$

During training, the network gradually learns better weights through optimisation.

In transfer learning, however, the network begins with pretrained weights:

$$W_{\text{pretrained}}$$

These weights already encode meaningful visual representations.

Fine-tuning then modifies the weights slightly to better match the new dataset.

$$W_{\text{new}} = W_{\text{pretrained}} + \Delta W$$

Where:

- ΔW represents the change applied during training
- This change is computed using gradient descent.

Because the starting point is already strong, the required adjustments are typically small.

4. Loss Function (Training Objective)

A neural network must be given a numerical objective that measures how good or bad its predictions are. This objective is called the **loss function**.

For classification tasks, the most commonly used loss function is **cross-entropy loss**.

Mathematically:

$$L = - \sum y \log(\hat{y})$$

Where:

- y is the true label
- $\hat{y}$ is the predicted probability

Cross-entropy measures how far the predicted probability distribution is from the true distribution.

Consider the following example:

True class: melanoma

Predicted probabilities:

Class	Probability
melanoma	0.8
nevus	0.1
benign	0.1

The loss becomes:

$$L = -\log(0.8)$$

Because the correct class has a high probability, the loss is relatively small.

If the model predicted incorrectly:

Class	Probability
melanoma	0.1
nevus	0.7
benign	0.2

Then the loss becomes:

$$L = -\log(0.1)$$

This produces a large loss value, which signals the model to adjust its weights.

5. Gradient Descent Optimisation

The process of updating the network's weights to minimise the loss is called **gradient descent**.

The update rule is given by:

$$W = W - \eta \nabla L$$

Where:

- W represents model weights
- η (eta) is the learning rate
- ∇L is the gradient of the loss with respect to the weights

The gradient indicates the direction in which the loss increases the fastest. By moving in the opposite direction, the model reduces the loss.

The update process looks like this:

Compute predictions

↓

Compute loss

↓

Compute gradient

↓

Update weights

↓

Repeat

In transfer learning, the learning rate is usually **lower** than normal because the pretrained weights are already close to optimal.

Large updates could destroy the useful information learned during pretraining.

6. Freezing Layers

In many transfer learning scenarios, some layers of the model are **frozen** so that their weights do not change during training.

In PyTorch this is implemented using:

```
weights.requires_grad = False
```

When this property is set to false:

- gradients are not computed
- weights are not updated during backpropagation

Mathematically, this means:

$$\Delta W = 0$$

for frozen layers.

Freezing layers is useful because:

- Early layers already contain general visual knowledge

- Updating them might degrade that knowledge
- It reduces computational cost

Usually, only the **final classification layers** are trained.

7. Ensemble Learning (Core Idea)

A single model can sometimes produce unstable predictions due to random initialisation, dataset noise, or architectural limitations.

Ensemble learning addresses this problem by combining predictions from multiple models.

Instead of relying on one model, several models independently make predictions, and their outputs are combined to produce a final result.

```
Model A → prediction
Model B → prediction
Model C → prediction
      ↓
Combine predictions
      ↓
Final decision
```

This approach improves reliability because errors made by one model may be corrected by others.

In your project, the ensemble consists of multiple modern architectures, such as:

- EfficientNetV2M
- EfficientNetV2S
- ConvNeXt

Each model analyses the same input image but processes it differently due to differences in architecture.

8. Ensemble Mathematics

The simplest ensemble method is **probability averaging**.

Each model outputs a probability distribution over classes. The ensemble prediction is obtained by averaging these probabilities.

Mathematically:

$$P_{\text{ensemble}} = (1/N) \sum P_i$$

Where:

- N is the number of models

- P_i is the probability predicted by model i

Example:

Model	Melanoma Probability
Model A	0.7
Model B	0.6
Model C	0.8

The ensemble probability becomes:

$$P = (0.7 + 0.6 + 0.8) / 3$$

$$P = 0.7$$

The final prediction corresponds to the class with the highest averaged probability.

9. Weighted Ensemble

Sometimes, some models are more accurate than others.

In that case, we assign **weights** to models so that stronger models contribute more to the final prediction.

$$P = w_1P_1 + w_2P_2 + w_3P_3$$

Where:

- w_1, w_2, w_3 are model weights
- P_1, P_2, P_3 are model predictions

Example weighting scheme:

Model	Weight
EfficientNet	0.5
ConvNeXt	0.3
ResNet	0.2

The final probability becomes a weighted combination of predictions.

10. Bias-Variance Tradeoff

The total prediction error of a model can be decomposed as:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Where:

- **Bias** measures how far predictions are from the true function
- **Variance** measures how sensitive the model is to training data fluctuations
- **Noise** represents unavoidable randomness in the data

Ensemble learning primarily reduces **variance**.

Instead of relying on one potentially unstable model, the ensemble averages multiple predictions, which stabilizes the final output.

Single Model → high variance

Multiple Models → averaged variance

11. Softmax Function

Neural networks do not directly output probabilities. Instead, they produce raw scores called **logits**.

These logits must be converted into probabilities using the **softmax function**.

Mathematically:

$$P_i = \exp(z_i) / \sum \exp(z_j)$$

Where:

- z_i is the logit for class i
- the denominator sums the exponentials of all logits

Example logits:

```
melanoma = 3  
nevus = 1  
benign = 0
```

Applying softmax produces probabilities:

```
melanoma = 0.84  
nevus = 0.11  
benign = 0.05
```

These probabilities sum to 1 and represent the model's confidence in each class.

The probabilities produced by softmax are what are combined during ensemble averaging.

12. Application to the Skin Cancer Classification Project

In this project, the dataset used is **HAM10000**, which contains thousands of dermoscopic images of skin lesions.

The pipeline follows this structure:

```
Dataset
  ↓
Image preprocessing
  ↓
Transfer learning models
  ↓
Fine-tuning
  ↓
Softmax probabilities
  ↓
Ensemble averaging
  ↓
Final prediction
```

Each architecture learns slightly different visual representations of skin lesions, which improves diagnostic accuracy when their predictions are combined.

13. Conceptual Interpretation

A helpful way to visualise ensemble learning is to imagine a **panel of dermatologists** analyzing the same skin lesion.

```
Doctor 1 → EfficientNet
Doctor 2 → ConvNeXt
Doctor 3 → EfficientNetV2S
```

Each doctor provides a probability estimate of the diagnosis. The final decision is based on the combined opinion of the entire panel.

This collaborative approach reduces the risk of individual errors and leads to a more reliable prediction.